

SQL

Operators & Clauses

Operators

Wildcards

Alias

LIKE

IS NULL

AND/OR/NOT

DISTINCT

01

Operators

Arithmetic, Comparison, Logical & Bitwise

SQL Operators – Overview

Arithmetic

- + (Add)
- - (Subtract)
- * (Multiply)
- / (Divide)
- % (Modulo)

Comparison

- = Equal
- != / <> Not equal
- > Greater than
- < Less than
- >= / <=

Logical

- AND
- OR
- NOT
- IN
- BETWEEN
- LIKE
- EXISTS

Bitwise

- & (AND)
- | (OR)
- ^ (XOR)
- ~ (NOT)
- >> Shift right
- << Shift left

SQL Operators – Live Examples

-- ① ARITHMETIC OPERATORS

```
SELECT ProductName,  
       Price,  
       Price * 1.18 AS PriceWithGST,  
       Price * 0.9  AS Discounted10Pct,  
       Price % 100  AS Remainder  
FROM   Products;
```

-- ② COMPARISON OPERATORS

```
SELECT * FROM Employees  
WHERE Salary >= 50000 -- greater than or equal  
       AND Department <> 'HR'; -- not equal to HR
```

-- ③ COMBINATION: Arithmetic + Comparison

```
SELECT EmpName,  
       Salary,  
       Salary * 12 AS AnnualSalary,  
       Salary * 12 * 0.10 AS AnnualBonus  
FROM   Employees  
WHERE  Salary * 12 > 600000; -- annual > 6 lakh
```

Result Preview

EmpName	Salary	AnnualSal
Rahul	55000	660000
Priya	72000	864000
Arjun	52000	624000



Arithmetic operators work in SELECT, WHERE & ORDER BY. Use parentheses () to control priority.

Operator Precedence

Highest → Lowest:

1. () Parentheses
2. * / %
3. + -
4. = != < > <= >=
5. NOT
6. AND
7. OR ← lowest

02

LIKE Operator

Pattern-based filtering in WHERE clause

LIKE Operator – Pattern Matching

Syntax:

```
SELECT column(s) FROM table WHERE column LIKE 'pattern';
```

'A%'

Starts with A

e.g. Arjun, Amit, Avni

'%Kumar'

Ends with Kumar

e.g. Raj Kumar, Sam Kumar

'%an%'

Contains 'an' anywhere

e.g. Priyanka, Anita, Rani

'R____'

R + exactly 3 more characters

e.g. Riya, Ravi, Rani

'[AP]%'

Starts with A or P

e.g. Amit, Priya, Arjun

'[^S]%'

Does NOT start with S

e.g. Rahul, Priya, Kiran

💡 Use NOT LIKE to exclude patterns: WHERE Name NOT LIKE 'R%' → excludes names starting with R.

LIKE – Deep Dive with Real Queries

```
-- Search customer emails from gmail
SELECT CustomerName, Email
FROM Customers
WHERE Email LIKE '%@gmail.com';

-- Find product codes: start with 'PRD', 3-digit number
SELECT ProductCode, ProductName
FROM Products
WHERE ProductCode LIKE 'PRD[0-9][0-9][0-9]';

-- Phone numbers starting with 98 or 99
SELECT Name, Phone
FROM Contacts
WHERE Phone LIKE '9[89]%';

-- Escape special character: find 50% discount items
-- Use ESCAPE to treat % as literal
SELECT ProductName
FROM Products
WHERE Description LIKE '%50!%%' ESCAPE '!';
```

```
-- NOT LIKE: exclude test data
SELECT * FROM Users
WHERE Email NOT LIKE '%test%'
AND Email NOT LIKE '%dummy%';
```

LIKE vs = (Equals)

Feature	LIKE	=
Pattern match	✓ Yes	✗ No
Exact match	✓ Yes	✓ Yes
Wildcards	✓ Yes	✗ No
Performance	Slower	Faster
Case sensitive	Depends	Depends



For EXACT matches always prefer = over LIKE.
It's faster because SQL can use indexes.

```
-- ✓ Fast: uses index
WHERE Email = 'user@gmail.com'
```

```
-- 🐢 Slow: cannot use index
WHERE Email LIKE '%gmail.com'
```

03

AND / OR / NOT

Combining multiple conditions in WHERE clause

AND, OR, NOT – Logical Operators

AND

Both conditions must be TRUE

```
TRUE AND TRUE → TRUE  
TRUE AND FALSE → FALSE  
FALSE AND FALSE → FALSE
```

```
SELECT * FROM Employees  
WHERE Department = 'IT'  
    AND Salary > 50000;
```

OR

At least ONE condition must be TRUE

```
TRUE OR FALSE → TRUE  
FALSE OR FALSE → FALSE  
TRUE OR TRUE → TRUE
```

```
SELECT * FROM Employees  
WHERE City = 'Surat'  
    OR City = 'Mumbai';
```

NOT

Reverses / negates the condition

```
NOT TRUE → FALSE  
NOT FALSE → TRUE  
NOT NULL → NULL
```

```
SELECT * FROM Employees  
WHERE NOT Department = 'HR';  
-- same as: Department <> 'HR'
```

💡 Precedence: NOT > AND > OR. Always use () parentheses when mixing AND with OR to avoid logic bugs!

AND / OR / NOT – Advanced Combinations

```
-- Complex condition with parentheses
SELECT EmpName, Department, Salary, City
FROM Employees
WHERE (Department = 'IT' OR Department = 'Finance')
      AND Salary > 50000
      AND City <> 'Delhi';
```

```
-- NOT with BETWEEN
SELECT ProductName, Price
FROM Products
WHERE Price NOT BETWEEN 100 AND 500;
```

```
-- NOT with IN
SELECT EmpName, Department
FROM Employees
WHERE Department NOT IN ('HR', 'Admin', 'Legal');
```

```
-- NOT with LIKE
SELECT EmpName, Email
FROM Employees
WHERE Email IS NOT NULL
      AND Email NOT LIKE '%test%';
```

-- ⚠ Parentheses matter! Different results:

```
-- Version A (wrong intent)
WHERE Dept = 'IT' OR Dept = 'HR' AND Salary > 50000
```

Truth Table Quick Reference

A	B	A AND B	A OR B
TRUE	TRUE	TRUE	TRUE
TRUE	FALSE	FALSE	TRUE
FALSE	TRUE	FALSE	TRUE
FALSE	FALSE	FALSE	FALSE

NOT Operator

Condition	NOT Result
TRUE	FALSE
FALSE	TRUE
NULL	NULL



Replace long OR chains with IN(): City='Surat'
OR City='Mumbai' → City IN ('Surat','Mumbai')

04 Wildcards

Pattern characters used with LIKE operator

Wildcards – Pattern Matching Characters

%

Percent

Zero or more characters

Example:

'R%' → Rahul, Riya, Ram

_

Underscore

Exactly ONE character

Example:

'_iya' → Riya, Miya

[]

Bracket Set

Any ONE char in the set

Example:

'[RSM]iya' → Riya, Siya, Miya

[^]

Negated Set

Any char NOT in the set

Example:

'[^RS]%' → Mona, Priya



Wildcards only work with the LIKE operator. Using = with % treats % as a literal character!

Wildcards – Practice Queries

```
-- % : Find all employees whose name STARTS with 'R'
SELECT * FROM Employees WHERE Name LIKE 'R%';

-- % : Names ENDING with 'a'
SELECT * FROM Employees WHERE Name LIKE '%a';

-- % : Names CONTAINING 'ar' anywhere
SELECT * FROM Employees WHERE Name LIKE '%ar%';

-- _ : Names with exactly 4 letters starting with 'R'
SELECT * FROM Employees WHERE Name LIKE 'R___';

-- Combining wildcards: 2nd letter is 'i'
SELECT * FROM Employees WHERE Name LIKE '_i%';
```

```
-- [] : Name starts with R, S, or P
SELECT * FROM Employees
WHERE Name LIKE '[RSP]%;

-- [^] : Name does NOT start with R or S
SELECT * FROM Employees
WHERE Name LIKE '[^RS]%;
```

Match / No-Match Table

Pattern	Matches?	Example
'R%'	✓ YES	Rahul, Ram
'%hu'	✓ YES	Rahu→l? No – 'Raghu'
'R___'	✓ YES	Riya, Rama
'R_ '	✗ NO	Rahul (>2 chars)
'[RS]%'	✓ YES	Riya, Sita
'[^R]%'	✓ YES	Priya, Amit



In SQL Server, wildcards in LIKE are case-insensitive by default (depends on collation).

05

Alias

Giving temporary names to columns and tables

Alias – AS Keyword

Column Alias

Renames a column in the result set output.

```
-- With AS keyword (recommended)
SELECT FirstName AS "First Name",
       LastName AS Surname,
       Salary * 12 AS AnnualSalary,
       GETDATE() AS TodayDate
FROM Employees;

-- Without AS (also valid)
SELECT FirstName "First Name",
       Salary * 12 AnnualPay
FROM Employees;
```

Table Alias

```
-- Table alias (very useful in JOINS)
SELECT e.EmpName, d.DeptName
FROM Employees AS e
JOIN Departments AS d
ON e.DeptID = d.DeptID;
```

Result Preview

First Name	Surname	AnnualSalary
Rahul	Shah	720000
Priya	Patel	864000
Arjun	Mehta	624000

Rules for Alias

- Use double quotes for spaces: AS "First Name"
- Alias visible only in ORDER BY, not in WHERE
- Alias cannot be used in GROUP BY (use column directly)
- Table alias shortens long table names in joins
- AS keyword is optional but recommended for clarity

06

IS NULL / IS NOT NULL

Handling missing values in SQL

IS NULL – Handling Missing Data

NULL means UNKNOWN or MISSING. It is NOT zero, NOT an empty string – it literally has no value.

```
-- Find employees with NO email address
SELECT EmpName, Department
FROM Employees
WHERE Email IS NULL;

-- Find employees WHO HAVE an email
SELECT EmpName, Email
FROM Employees
WHERE Email IS NOT NULL;

-- Replace NULL with a default using ISNULL()
SELECT EmpName,
       ISNULL(Email, 'Not Provided') AS Email
FROM Employees;

-- COALESCE: returns first non-NULL value
SELECT EmpName,
       COALESCE(Mobile, Phone, 'No Contact') AS Contact
FROM Employees;
```

Employees Table

EmpName	Email	Mobile
Rahul	r@g.com	9876543210
Priya	NULL	9988776655
Arjun	a@g.com	NULL
Neha	NULL	NULL

✗ Common Mistakes with NULL

```
-- ✗ WRONG: = NULL never works!
WHERE Email = NULL      -- always FALSE

-- ✗ WRONG: != NULL never works!
WHERE Email != NULL     -- always FALSE

-- ✓ CORRECT:
WHERE Email IS NULL
```

07

DISTINCT

Removing duplicate rows from results

DISTINCT – Unique Values Only

DISTINCT removes duplicate rows from the SELECT result. It operates on the entire row, not just one column.

```
-- Syntax
SELECT DISTINCT column1, column2 FROM table;

-- ❶ Single column: unique departments
SELECT DISTINCT Department FROM Employees;

-- ❷ Multiple columns: unique city+dept combos
SELECT DISTINCT Department, City
FROM Employees;

-- ❸ With WHERE clause
SELECT DISTINCT City
FROM Employees
WHERE Department = 'IT';

-- ❹ COUNT DISTINCT: how many unique departments?
SELECT COUNT(DISTINCT Department) AS UniqueDepts
FROM Employees;

-- ❺ DISTINCT with ORDER BY
SELECT DISTINCT City
FROM Employees
```

Without DISTINCT

Department
IT
HR
IT
Sales
HR
IT
Finance

With DISTINCT

Department
Finance
HR
IT

DISTINCT – Advanced Usage & Pitfalls

```
-- ⚠️ DISTINCT on multiple columns:
-- removes rows where ALL columns are identical
SELECT DISTINCT FirstName, LastName
FROM Employees;
-- "Rahul Shah" and "Rahul Patel" are KEPT (different LastName)
-- Two "Rahul Shah" rows → only ONE kept

-- ✅ COUNT DISTINCT: count unique values
SELECT COUNT(DISTINCT City) AS UniqueCities,
       COUNT(DISTINCT Department) AS UniqueDepts,
       COUNT(*) AS TotalEmployees
FROM Employees;

-- ✅ DISTINCT with aggregate functions
SELECT Department,
       COUNT(DISTINCT City) AS CitiesPerDept
FROM Employees
GROUP BY Department;
```

```
-- ❌ You CANNOT use DISTINCT inside aggregates like this:
SELECT SUM(DISTINCT Salary) FROM Employees;
-- This WORKS but usually not what you want
-- It sums only unique salary values (rare use-case)

-- ✅ Better: use GROUP BY for grouping
SELECT Department, COUNT(*) AS Headcount
FROM Employees
```

DISTINCT vs GROUP BY

Feature	DISTINCT	GROUP BY
Purpose	Unique rows	Group + aggregate
Aggregates	Limited	Full support
Performance	Similar	Similar
Use case	Unique list	Summary stats
ORDER BY	Optional	Optional

💡 Use DISTINCT for unique lists. Use GROUP BY when you also need COUNT, SUM, AVG on each group.

```
-- Same result, different approach:
SELECT DISTINCT City FROM Employees;
SELECT City FROM Employees GROUP BY City;
```

Master Cheat Sheet – All Topics

Topic	Syntax	Key Rule
Operators	<code>+ - * / % = != > <</code>	Arithmetic before comparison; use <code>()</code> to control order
Wildcards	<code>% _ [] [^]</code>	Use only with <code>LIKE</code> ; <code>%</code> = many chars, <code>_</code> = exactly one
Alias	<code>col AS 'Name'</code>	Spaces need double quotes; not usable in <code>WHERE</code>
LIKE	<code>col LIKE 'pattern'</code>	Slower than <code>=</code> ; <code>NOT LIKE</code> to exclude; <code>ESCAPE</code> for literals
IS NULL	<code>col IS NULL</code>	Never use <code>= NULL</code> or <code>!= NULL</code> — they always return false
IS NOT NULL	<code>col IS NOT NULL</code>	<code>ISNULL(col,'default')</code> replaces <code>NULL</code> with a value
AND	<code>cond1 AND cond2</code>	Both must be true; higher precedence than <code>OR</code>
OR	<code>cond1 OR cond2</code>	At least one true; use <code>()</code> to mix with <code>AND</code> safely
NOT	<code>NOT condition</code>	Reverses result; <code>NOT NULL</code> is still <code>NULL</code>
DISTINCT	<code>SELECT DISTINCT col</code>	Removes duplicate rows; works on entire selected row

Quick Practice Quiz

Can you write the queries?

Q1. Find all employees whose name CONTAINS the letter 'a'

Answer: `WHERE Name LIKE '%a%'`

Q2. Show unique cities where IT department employees live

Answer: `SELECT DISTINCT City FROM Employees WHERE Department = 'IT'`

Q3. Find employees with NO phone number AND NO email

Answer: `WHERE Phone IS NULL AND Email IS NULL`

Q4. List products costing between 100-500, NOT in 'Electronics'

Answer: `WHERE Price BETWEEN 100 AND 500 AND Category NOT LIKE 'Electronics'`

Q5. Employees from Surat or Mumbai earning above 40000

Answer: `WHERE (City = 'Surat' OR City = 'Mumbai') AND Salary > 40000`

Keep Coding.

Keep Querying.

Operators • Wildcards • Alias • LIKE • IS NULL • AND/OR/NOT • DISTINCT

SQL Server — Dark Edition — Practice daily in SSMS!